



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC2213 — Lógica para Ciencia de la Computación — 2024-01

Examen

Duración: 2:30 hrs.

Observación: Cada pregunta tiene una nota de 1 a 7 (hay un punto base). La nota final es el promedio de las 3 preguntas. Dentro de cada pregunta, cada ítem vale lo mismo.

Pregunta 1

Sea $\mathcal{L}$ un vocabulario arbitrario. Para cada una de las siguientes propiedades, escriba una $\mathcal{L}$ -oración en lógica de primer orden que exprese la propiedad:

1. $\mathcal{P}_1 = \{\mathfrak{A} \in S[\mathcal{L}] \mid \text{la cantidad de elementos en el dominio de } \mathfrak{A} \text{ es mayor o igual a } 3\}.$
2. $\mathcal{P}_2 = \{\mathfrak{A} \in S[\mathcal{L}] \mid \text{la cantidad de elementos en el dominio de } \mathfrak{A} \text{ es menor o igual a } 3\}.$
3. $\mathcal{P}_3 = \{\mathfrak{A} \in S[\mathcal{L}] \mid \text{la cantidad de elementos en el dominio de } \mathfrak{A} \text{ es igual a } 3\}.$

(Recuerde que $S[\mathcal{L}]$ denota el conjunto de todas las $\mathcal{L}$ -estructuras.)

Solución

Para la primera propiedad, se necesita que el dominio de la estructura contenga tres elementos que no son iguales entre ellos, por lo tanto una oración que expresa $\mathcal{P}_1$ podría ser

$$\varphi_1 = \exists x_1 \exists x_2 \exists x_3 \bigwedge_{\{i,j\} \in \{1,2,3\}} \neg(x_i = x_j)$$

La segunda propiedad es equivalente a decir que para toda elección de 4 elementos del dominio de $\mathfrak{A}$ se tiene que al menos dos de ellos son iguales, es decir, una oración que captura $\mathcal{P}_2$ es

$$\varphi_2 = \forall x_1 \forall x_2 \forall x_3 \forall x_4 \bigvee_{\{i,j\} \in \{1,2,3,4\}} x_i = x_j$$

Finalmente, la última propiedad es la intersección de las dos propiedades anteriores, por lo que una oración que la exprese podría ser

$$\varphi_3 = \varphi_1 \wedge \varphi_2$$

Se da dos puntos por cada una de las oraciones, no es necesario que justifiquen por qué funcionan. Pueden usar la fórmula de una propiedad para las otras como se muestra en el ejemplo de la pauta.

Pregunta 2

1. Sea $\mathcal{L}$ un vocabulario, y $\mathfrak{A}, \mathfrak{B}$ dos $\mathcal{L}$ -estructuras arbitrarias. Definimos el siguiente conjunto de $\mathcal{L}$ -oraciones:

$$\text{Th}(\mathfrak{A}, \mathfrak{B}) = \{\varphi \mid \varphi \text{ es una } \mathcal{L}\text{-oración tal que } \mathfrak{A} \models \varphi \text{ y } \mathfrak{B} \models \varphi\}.$$

Demuestre que $\text{Th}(\mathfrak{A}, \mathfrak{B})$ es una teoría.

2. Demuestre que si $\mathfrak{A} = \mathfrak{B}$, entonces $\text{Th}(\mathfrak{A}, \mathfrak{B})$ es una teoría completa.

(Ojo: En clases vimos que la teoría de una estructura es completa. Si quiere usar ese resultado, **debe** demostrarlo.)

Solución

Parte 1

Para probar que un conjunto de oraciones es teoría hay que probar ambos puntos de la definición

1. $\text{Th}(\mathfrak{A}, \mathfrak{B})$ es satisfacible: Notemos que si $\varphi \in \text{Th}(\mathfrak{A}, \mathfrak{B})$, entonces $\mathfrak{A} \models \varphi$. Por lo tanto, el conjunto $\text{Th}(\mathfrak{A}, \mathfrak{B})$ es satisfacible por $\mathfrak{A}$.
2. $\text{Th}(\mathfrak{A}, \mathfrak{B})$ es cerrado bajo conclusión lógica: Supongamos que $\text{Th}(\mathfrak{A}, \mathfrak{B}) \models \varphi$, entonces como $\mathfrak{A} \models \text{Th}(\mathfrak{A}, \mathfrak{B})$ y $\mathfrak{B} \models \text{Th}(\mathfrak{A}, \mathfrak{B})$, también tenemos que $\mathfrak{A} \models \varphi$ y $\mathfrak{B} \models \varphi$. Por lo tanto, $\varphi \in \text{Th}(\mathfrak{A}, \mathfrak{B})$ y el conjunto es cerrado bajo conclusión lógica.

Por lo tanto, podemos concluir que $\text{Th}(\mathfrak{A}, \mathfrak{B})$ es teoría.

Parte 2

Si $\mathfrak{A} = \mathfrak{B}$, entonces tenemos que $\mathfrak{A} \models \varphi$ si y solo si $\mathfrak{B} \models \varphi$. Por lo tanto,

$$\text{Th}(\mathfrak{A}, \mathfrak{B}) = \{\varphi : \varphi \text{ es una } \mathcal{L}\text{-oración tal que } \mathfrak{A} \models \varphi\}$$

Es decir, $\text{Th}(\mathfrak{A}, \mathfrak{B}) = \text{Th}(\mathfrak{A})$. Por otro lado, para toda φ oración, se tiene que $\mathfrak{A} \models \varphi$ o $\mathfrak{A} \not\models \varphi$, por lo tanto $\mathfrak{A} \models \varphi$ o $\mathfrak{A} \models \neg\varphi$, de lo que podemos concluir que $\varphi \in \text{Th}(\mathfrak{A})$ o $\neg\varphi \in \text{Th}(\mathfrak{A})$. Es decir, $\text{Th}(\mathfrak{A})$ es una teoría completa y por ende $\text{Th}(\mathfrak{A}, \mathfrak{B})$ es completa.

Cada parte tiene 3 puntos. En la primera, se da 1,5 puntos por cada punto de la definición. En la segunda, se da 1 punto por concluir que $\text{Th}(\mathfrak{A}, \mathfrak{B}) = \text{Th}(\mathfrak{A})$ y los otros 2 puntos por argumentar que la teoría de una estructura es completa.

Pregunta 3

Dado un grafo $G = (V, E)$ decimos que un subconjunto de nodos $I \subseteq V$ es un *conjunto independiente* si:

para todo par de nodos distintos $u, v \in I$ se cumple que $\{u, v\} \notin E$.

Es decir, en un conjunto independiente *ningún* par de nodos está conectado por una arista.

Definimos el siguiente problema:

$\text{INDEP} = \{(G, k) \mid G \text{ es un grafo, } k \geq 1, \text{ y } G \text{ contiene un conjunto independiente } I \text{ tal que } |I| = k\}$.

Demuestre que INDEP es NP-completo.

(*Hint*: para el NP-hardness utilice CLIQUE.)

Solución

Para probar lo pedido se necesitan dos cosas, que el lenguaje esté en NP y que sea NP-completo.

INDEP $\in$ NP

Para probar que INDEP está en NP se puede construir una máquina no determinista de dos cintas que lo acepte. Esta funcionará de la siguiente manera dado un input (G, k) (en caso de estar en otro formato se rechaza):

1. En la segunda cinta la máquina escribe k vértices diferentes de manera no determinista.
2. La máquina verifica si para cada par de vértices en la segunda cinta, hay una arista que los une. Si es así rechaza y si recorre todos los pares sin rechazar, acepta.

Esta máquina es no determinista y trabaja en tiempo polinomial, ya que en el primer paso solo necesita recorrer los vértices dos veces y en el segundo paso recorre las aristas una vez por cada par de vértices. Además acepta INDEP, ya que para todo grafo aceptado existe un conjunto de k vértices (el que se escribió en la segunda cinta) que forman un conjunto independiente.

Por lo tanto, tenemos que INDEP está en NP.

INDEP es NP-hard

Para probar esto se presentará una reducción desde CLIQUE. Definimos la reducción $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ de la siguiente manera:

1. Si z no tiene la forma $z = (G, k)$ entonces $f(z) = 0 \notin \text{INDEP}$.
2. Si $z = (G, k)$ con $G = (V, E)$, entonces definimos $G' = (V, E')$ donde E' es el conjunto de todos los pares (menos loops) que no están en E y $f(z) = (G', k)$

Esto es computable en tiempo polinomial, ya que solo requiere recorrer el conjunto de todos los pares de vértices, revisar si están o no en E y copiarlo si no es así.

Probemos ahora que es reducción, es decir, $z \in \text{CLIQUE}$ si y solo si $f(z) \in \text{INDEP}$.

($\Rightarrow$) Si $z \in \text{CLIQUE}$, entonces $z = (G, k)$ tal que G contiene un clique de tamaño k , es decir hay un conjunto $V' \subseteq V$ de k elementos tal que todo par de vértices está en E . Luego, tenemos que en G' ningún par de elementos de V' forma una arista, es decir, V' es un conjunto independiente de tamaño k y $f(z) = (G', k) \in \text{INDEP}$.

($\Leftarrow$) Si $f(z) \in \text{INDEP}$, entonces $f(z) = f(G, k) = (G', k)$ tal que G' contiene un conjunto de k vértices $V' \subseteq V$ tal que no hay arista entre ningún par de ellos. Luego, en G se tenían todas las aristas posibles entre los vértices de V' , es decir, V' forma un clique de tamaño k y $z \in \text{CLIQUE}$.

En esta pregunta se dan 2 puntos por probar que el lenguaje está en NP (puede ser con cualquiera de las dos definiciones vistas en clases) y 4 puntos por probar que es NP-hard. De los 4 puntos, se dan 2 por construir la reducción y 1 por cada lado de la doble implicancia. (El tema de los loops da igual para la reducción)